

LangChain Study Guide

A practical PDF for testing your PDF Reader / RAG application

This document covers the main LangChain concepts used in a basic LLM application, including prompts, output parsers, chains, runnables, document processing, embeddings, vector stores, retrievers, and RAG.

Topics Covered

- LangChain overview
- LLMs and chat models
- Prompt Templates
- String and structured output parsing
- Simple and sequential chains
- Parallel and conditional chains
- LCEL and Runnables
- PDF/document loaders
- Text splitting
- Embeddings
- Vector stores
- Retriever
- RAG architecture
- Example questions for testing a PDF chatbot

1. What is LangChain?

LangChain is a framework for building applications around language models. Instead of calling an LLM directly for every task, an application can combine prompts, models, parsers, retrievers, tools, and other components into reusable workflows.

- A LangChain application commonly follows a flow such as Prompt → Model → Parser.
- More advanced applications can add documents, embeddings, vector stores, and retrieval.
- RAG applications use external documents as context before asking the model to generate an answer.

2. LLMs and Chat Models

An LLM generates text based on an input prompt. A chat model is an interface designed around messages and conversational interactions. In your local setup, TinyLlama can be loaded through Hugging Face and connected to LangChain.

- The model is responsible for generation.
- The prompt tells the model what task to perform.
- The output parser converts the model output into a useful application format.

3. PromptTemplate

PromptTemplate allows you to create reusable prompts with variables. Instead of hard-coding a complete prompt each time, you define a template and provide values when invoking it.

- Example variable: {topic}.
- The same template can be reused for cricket, artificial intelligence, cybersecurity, or any other topic.
- Prompt templates are commonly used as the first component of an LCEL chain.

Example

```
prompt = PromptTemplate(
    template="Generate 5 interesting facts about {topic}.",
    input_variables=["topic"]
)
```

4. Output Parsers

An output parser processes the response returned by an LLM. A StrOutputParser is useful when you simply want the generated text as a string. A JSON parser can convert valid JSON text into a Python dictionary. Structured output approaches can enforce a desired set of fields, depending on the model and LangChain version.

- String output → useful for normal text.
- JSON output → useful when you need key/value data.

- Structured output → useful when an application expects a defined schema.

5. Simple Chain

A simple LCEL chain connects components with the pipe operator. The output of one component becomes the input of the next component.

- Typical pattern: prompt | model | parser.
- `invoke()` starts the chain.
- This makes the workflow easier to read and reuse.

Example

```
chain = prompt | model | parser
result = chain.invoke({"topic": "cricket"})
```

6. Sequential Chain

A sequential workflow performs one task and then uses its output as input for another task. For example, the first model call can generate a detailed report and the second can summarize that report.

- Prompt 1 → Model → Parser → Prompt 2 → Model → Parser.
- The output from the first stage becomes input to the next stage.
- Sequential workflows are useful when a large task can be divided into smaller steps.

7. Parallel Chain

A parallel workflow runs multiple branches using the same input. For example, the same topic can be sent to one branch that creates notes and another branch that creates a quiz.

- Input topic → Notes branch.
- Input topic → Quiz branch.
- The branches can produce independent results.

8. Conditional Chain

A conditional chain chooses a branch based on an input or an intermediate result. A common example is feedback handling: positive feedback can produce a thank-you response, while negative feedback can produce an apology and improvement message.

- Input → condition.
- If condition is positive → positive response.
- If condition is negative → negative-feedback response.

9. Runnables and LCEL

LangChain Expression Language (LCEL) provides a composable way to connect runnable components. Prompts, models, parsers, retrievers, and other components can be combined into pipelines.

- The pipe symbol | connects runnable components.
- `invoke()` runs the workflow for one input.
- `get_graph()` can be used to inspect a chain's structure when the required graph-visualization dependency is installed.

10. PDF Document Loader

A PDF loader reads a PDF and converts its pages into LangChain Document objects. A common loader is `PyPDFLoader`.

- Each document contains page text.
- Metadata can include information such as the page number.
- The loaded documents can then be split into smaller chunks.

Example

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("sample.pdf")
documents = loader.load()
```

11. Text Splitter

Large documents are usually divided into smaller chunks before embedding. `RecursiveCharacterTextSplitter` is a common choice.

- `chunk_size` controls the approximate size of each chunk.
- `chunk_overlap` keeps some text from neighboring chunks so useful context is less likely to be lost.
- Smaller chunks can make retrieval more focused, while larger chunks can preserve more context.

12. Embeddings

An embedding model converts text into numerical vectors. Texts with related meanings tend to have vectors that are closer in the embedding space.

- Embeddings are used for semantic search.
- A Hugging Face sentence-transformer can generate embeddings locally.

- The same embedding model should generally be used consistently when building and querying a vector store.

Example

```
embeddings = HuggingFaceEmbeddings(  
    model_name="sentence-transformers/all-MiniLM-L6-v2"  
)
```

13. Vector Store

A vector store keeps document embeddings and supports similarity search. FAISS is one option for local vector storage.

- Documents are converted into embeddings.
- The embeddings are stored in the vector database.
- A query is embedded and compared with stored vectors to find relevant documents.

14. Retriever

A retriever searches the vector store for information relevant to a user's question. For example, a retriever can return the top three most relevant PDF chunks.

- The question is used as the search query.
- The retriever returns relevant document chunks.
- Those chunks become context for the LLM.

Example

```
retriever = vector_store.as_retriever(  
    search_kwargs={"k": 3}  
)  
  
docs = retriever.invoke(question)
```

15. RAG — Retrieval-Augmented Generation

RAG combines retrieval with generation. Instead of asking the LLM to answer only from its internal knowledge, the application retrieves relevant information from a document collection and supplies that information as context.

- PDF → Loader → Documents.
- Documents → Splitter → Chunks.
- Chunks → Embeddings → Vector Store.
- Question → Retriever → Relevant Chunks.
- Relevant Chunks + Question → LLM → Answer.

- LLM Answer → Output Parser → Final Response.

16. Complete PDF RAG Architecture

The complete architecture can be understood as two major phases. The indexing phase prepares the PDF for search. The query phase retrieves relevant content and generates an answer.

- INDEXING: PDF → Loader → Splitter → Embeddings → Vector Store.
- QUERY: Question → Retriever → Relevant Context → Prompt → LLM → Parser → Answer.
- This separation makes it possible to reuse an indexed document collection for many questions.

17. Example RAG Prompt

A RAG prompt should clearly tell the model to use the retrieved context. It can also instruct the model what to do when the answer is not present in the retrieved information.

- Context: {context}
- Question: {question}
- Instruction: Answer using the supplied context.
- Fallback: Say that the answer is not available when the context does not contain the answer.

18. Questions to Test Your PDF Reader

After loading this PDF into your RAG application, try asking these questions:

- What is LangChain?
- What is the purpose of a PromptTemplate?
- What is the difference between a string parser and JSON output?
- What is a sequential chain?
- How does a parallel chain work?
- What is a conditional chain?
- What is an embedding?
- Why do we split documents into chunks?
- What is a vector store?
- What does a retriever do?
- What is RAG?
- Explain the complete PDF RAG pipeline.

- What happens during the indexing phase?
- What happens when a user asks a question?

19. Quick Revision

Remember the pipeline:

- PDF → Load → Split → Embed → Store → Retrieve → Prompt → LLM → Parse → Answer.
- PromptTemplate prepares the input.
- The LLM generates the response.
- OutputParser processes the response.
- Retriever finds relevant document information.
- RAG supplies retrieved information to the LLM before generation.